# IME 775 — Lecture 17

## Loss Functions, Softmax, and Stochastic Gradient Descent

---

## 1. The Training Objective

Recall from Chapter 8 that training a neural network means finding weights $\vec{w}$ and biases $\vec{b}$ that minimize a **loss function** measuring how far the network's predictions are from the ground truth.

$$L\left(\vec{w}, \vec{b}\right) = \sum_{i=0}^{n-1} L^{(i)}\left(\vec{y}^{(i)}, \bar{y}^{(i)}\right)$$

where:

- $n$ = number of training examples
- $\vec{y}^{(i)} = f(\vec{x}^{(i)}; \vec{w}, \vec{b})$ = network output (prediction) on input $i$
- $\bar{y}^{(i)}$ = ground truth for input $i$
- $L^{(i)}$ = per-example loss

Different tasks require different loss functions. Chapter 8 used MSE (regression loss). This chapter introduces the loss functions used for **classification** and other specialized tasks.

---

## 2. Regression Loss (L2 Loss) — Review

$$L^{(i)} = \|\vec{y}^{(i)} - \bar{y}^{(i)}\|^2 = \sum_{j=0}^{N-1} \left(y_j^{(i)} - \bar{y}_j^{(i)}\right)^2$$

This is the squared Euclidean distance between prediction and target. We studied this in Chapter 8 (with the $\frac{1}{2}$ convenience factor).

**When to use:** Continuous-valued outputs (regression tasks).

```
import torch
import torch.nn as nn

y_pred = torch.tensor([2.5, -1.0, 0.5, -0.5])
y_gt = torch.tensor([3.0, -1.0, 0.0, 0.5])

loss_fn = nn.MSELoss(reduction='sum')
loss = loss_fn(y_pred, y_gt)
print(f"MSE loss: {loss.item()}")  # 0.25 + 0 + 0.25 + 1.0 = 1.5
(then /4 for mean)
```

---

## 3. Cross-Entropy Loss

### The Classification Setup

For classification with $N$ classes, the ground truth is a **one-hot vector**:

$$\bar{y}^{(i)} = [0, \ldots, 0, \underbrace{1}_{j^*}, 0, \ldots, 0]$$

where $j^*$ is the correct class index. The prediction $\vec{y}^{(i)}$ is a probability vector where each element $y_j^{(i)}$ represents the predicted probability of class $j$. All elements sum to 1.

**Definition**

$$L^{(i)} = -\sum_{j=0}^{N-1} \bar{y}_j^{(i)} \log\left(y_j^{(i)}\right)$$

**Why It Works**

Since $\bar{y}^{(i)}$ is one-hot, all terms vanish except the one for the correct class $j^*$:

$$L^{(i)} = -\log\left(y_{j^*}^{(i)}\right)$$

| **Predicted prob of correct class** $y_{j^*}$ | **CE Loss** $-\log(y_{j^*})$ | **Interpretation** |
|---|---|---|
| 1.0 | 0 | Perfect — no loss |
| 0.9 | 0.105 | Good prediction |
| 0.5 | 0.693 | Uncertain |
| 0.1 | 2.303 | Bad prediction |
| 0.01 | 4.605 | Very bad |
| → 0 | → $\infty$ | Worst possible |

The loss is 0 when the correct class gets probability 1, and increases without bound as the probability of the correct class approaches 0.

*Workout:* A 4-class classifier produces prediction $\vec{y} = [0.1, 0.7, 0.15, 0.05]$ for a sample whose ground truth is class 1. Compute the CE loss.

**Solution:**

$$L = -\log(y_1) = -\log(0.7) \approx 0.357$$

Only the element at the GT class index (1) matters because the GT is one-hot $[0, 1, 0, 0]$.

### Binary Cross-Entropy (Two Classes)

When $N = 2$, we can represent the problem with a single scalar $y$ (probability of class 0):

$$L^{(i)} = -\bar{y}^{(i)} \log(y^{(i)}) - (1 - \bar{y}^{(i)}) \log(1 - y^{(i)})$$

This is the loss used for **binary classification**. The GT $\bar{y}$ is either 0 or 1.

```
y_pred = torch.tensor([0.8])
y_gt = torch.tensor([1.0])
bce_loss = nn.BCELoss()
loss = bce_loss(y_pred, y_gt)
print(f"Binary CE loss: {loss.item():.4f}")  # -log(0.8) ≈ 0.2231
```

---

## 4. Softmax: From Scores to Probabilities

### The Problem

A neural network classifier typically outputs a **score vector** $\vec{s} = [s_0, s_1, \ldots, s_{N-1}]$ where each score can be any real number (unbounded). We need to convert these to probabilities.

### The Softmax Function

$$\text{softmax}(\vec{s})_j = \frac{e^{s_j}}{\sum_{k=0}^{N-1} e^{s_k}}$$

**Properties:**

- Each element is in $(0, 1)$
- All elements sum to 1
- Higher scores → higher probabilities
- Differentiable everywhere

*Workout:* Compute the softmax of $\vec{s} = [2, 1, 0]$.

**Solution:**

$$e^2 = 7.389, \quad e^1 = 2.718, \quad e^0 = 1.000$$

$$S = 7.389 + 2.718 + 1.000 = 11.107$$

$$\text{softmax}(\vec{s}) = \left[\frac{7.389}{11.107}, \frac{2.718}{11.107}, \frac{1.000}{11.107}\right] = [0.665,\ 0.245,\ 0.090]$$

**Why "Softmax"?**

Softmax is a **smooth (differentiable) approximation** of the argmax-one-hot function:

| $\vec{s}$ | argmax-one-hot | softmax |
|---|---|---|
| $[9.99, 10]$ | $[0, 1]$ | $[0.4975, 0.5025]$ |
| $[10, 9.99]$ | $[1, 0]$ | $[0.5025, 0.4975]$ |

The argmax is discontinuous — a tiny change in scores causes a huge jump in the output. Softmax is continuous: similar scores produce similar probability vectors. This is essential for gradient-based training.

**Nuanced point — temperature scaling:** We can control the "sharpness" of softmax by dividing scores by a temperature $\tau$: $\text{softmax}(\vec{s}/\tau)$. As $\tau \to 0$, softmax approaches argmax (hard decisions). As $\tau \to \infty$, softmax becomes uniform (maximum uncertainty).

```
scores = torch.tensor([2.0, 1.0, 0.1, -1.0])
probs = torch.softmax(scores, dim=0)
print(f"Softmax: {probs}")  # [0.590, 0.217, 0.088, 0.029]
print(f"Sum: {probs.sum()}")  # 1.0
```

## 5. Softmax Cross-Entropy Loss

In practice, the last layer of a classifier network outputs raw scores, and we apply softmax + CE loss. PyTorch combines these into a single efficient operation:

```
scores = torch.tensor([[9.99, 10.0, 0.01, -10.0]])  # raw scores
(logits)
gt_class = torch.tensor([1])  # ground truth: class 1 (dog)

loss_fn = nn.CrossEntropyLoss()
loss = loss_fn(scores, gt_class)
print(f"Softmax CE loss: {loss.item():.4f}")
```

**Why combine them?** Two reasons:

1. **Numerical stability** — Computing $e^{s_j}$ for large scores can overflow; the combined operation uses the log-sum-exp trick to avoid this
2. **Convenience** — One function call instead of two

*Workout:* A 4-class image classifier (cat=0, dog=1, airplane=2, auto=3) outputs scores $\vec{s} = [9.99, 10, 0.01, -10]$. The image is actually a dog (class 1). Compute the softmax probabilities and the CE loss.

**Solution:**

Softmax: $[0.497, 0.502, 2.3 \times 10^{-5}, 1.0 \times 10^{-9}]$

CE loss: $-\log(0.502) \approx 0.689$

The network is not very confident — it gives nearly equal probability to cat and dog. The loss is relatively high (close to $\log 2 = 0.693$, which is the loss for a fair coin flip between two classes).

## 6. Focal Loss

### The Data Imbalance Problem

When some classes have far fewer training examples than others, the network focuses on "easy" examples (where it already does well) and ignores "hard" examples (where it does poorly).

### Definition

$$L = -(1 - y_t)^{\gamma} \log(y_t)$$

where $y_t$ is the predicted probability of the ground truth class, and $\gamma \geq 0$ is a focusing parameter.

| $y_t$ **(prob of correct class)** | **CE loss** $-\log(y_t)$ | **Focal (**$\gamma = 2$**)** $-(1 - y_t)^2 \log(y_t)$ |
|---|---|---|
| 0.9 | 0.105 | 0.00105 |
| 0.5 | 0.693 | 0.173 |
| 0.1 | 2.303 | 1.865 |

When $\gamma = 0$, focal loss = standard CE loss. As $\gamma$ increases, easy examples (high $y_t$) contribute dramatically less to the total loss, focusing training on hard examples.

---

## 7. Hinge Loss (Multi-class SVM Loss)

### Definition

For a training example with ground truth class $c$:

$$L = \sum_{j \neq c} \max(0,\ y_j - y_c + m)$$

where $m$ is a margin (usually $m = 1$).

**Intuition**

- If the correct class score exceeds all incorrect class scores by at least margin $m$ : **loss = 0** (stop improving)
- Otherwise: loss is proportional to the violation

**Key difference from CE loss:** Hinge loss is "lazy" — it stops pushing once the correct class wins by margin $m$. CE loss keeps pushing toward infinite confidence. This makes hinge loss useful when you care about correct classification but not about calibrated probabilities.

---

## 8. Stochastic Gradient Descent (SGD) and Minibatches

**The Computational Problem**

Computing the true gradient requires processing **all** $n$ training examples per iteration.

**The Solution: Minibatches**

Instead of computing the gradient over all examples, we sample a random subset called a **minibatch** (typically 32–256 examples) and compute the gradient only over that subset.

$$\nabla_{\vec{w}} L \approx \frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \nabla_{\vec{w}} L^{(i)}$$

where $\mathcal{B}$ is the minibatch.

**Why It Works**

- The minibatch gradient is a **noisy but unbiased estimate** of the true gradient
- Over many iterations, the noise averages out

**Key Concepts**

| Term | Definition |
|---|---|
| **Iteration** | One weight update using one minibatch |
| **Epoch** | One complete pass through all training data |
| **Batch size** | Number of examples per minibatch |
| **Learning rate decay** | Reducing $\eta$ after each epoch for finer convergence |

**Critical practice:** Randomly **shuffle** the training data after every epoch. Without shuffling, the network sees examples in the same order every epoch, which can lead to oscillations and poor convergence.

```
from torch.utils.data import DataLoader, TensorDataset

X = torch.randn(1000, 2)
Y = torch.randint(0, 3, (1000,))

dataset = TensorDataset(X, Y)
loader = DataLoader(dataset, batch_size=32, shuffle=True)

model = nn.Sequential(nn.Linear(2, 16), nn.ReLU(), nn.Linear(16,
3))
loss_fn = nn.CrossEntropyLoss()
optimizer = torch.optim.SGD(model.parameters(), lr=0.01)

for epoch in range(5):
    for X_batch, Y_batch in loader:
        scores = model(X_batch)
        loss = loss_fn(scores, Y_batch)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
```

---

## Key Takeaways

1. **Regression loss** (MSE/L2) measures squared distance — used for continuous outputs
2. **Cross-entropy loss** $-\log(y_{j^*})$ measures how confident the prediction is on the correct class
3. **Softmax** converts unbounded scores to probabilities: $\text{softmax}(s_j) = e^{s_j} / \sum e^{s_k}$
4. **Softmax + CE** is combined in PyTorch's `CrossEntropyLoss` for numerical stability
5. **Focal loss** down-weights easy examples to focus on hard ones (class imbalance)
6. **Hinge loss** stops improving once the correct class wins by a margin
7. Always **shuffle** training data between epochs